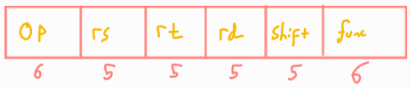
Machine Architecture

MIPS

Instruction types:

R-Type: register operands

Format



op: opcode - always 0 for R-type
rs, rt: source registers
rd: destination register
shift: Shift amount for shift operations, otherwise 0.
func: the function, serves as an opcode for R-type.

Example

add $s0, $s1, $s2 → 000000 | 10001 | 10010 | 10000 | 00000 | 100000

destination, source registers

OP $s1 $s2 $s0 shift add

adds the value in $s1 to the value in $s2, storing the result in $s0

I-Type: Immediate operands

Format

op: opcode
rs: source register
rt: target register
immediate: 16 bit twos complement binary number

Examples

addi $s0, $s1, 5 → 001000 | 10001 | 10000 | 0000 0000 0000 0101

target, source

addi $s1 $s0 5

add 5 to the value in $s1 and store the result in $s0

li $s0, 69 → ori $s0, $0, 69 → 001101 | 00000 | 10000 | 0000 0000 0100 0101

ori $0 $s0 69

implemented by performing OR on the zero register $0 and the number 69 then storing result in $s0.

load the number 69 into $s0

J-Type: Branching operations

format

Op: opcode
address: 26 bit address operand - converted to 32 bit address by compiler
→ The first four and last 2 bits are always 0 so they don't need to be stored.

Addressing Modes:

Register-only - uses registers for all source and destination operands
Used by R-type instructions

Immediate - uses registers and a 16-bit immediate as operands.
Used by some I-type instructions

Base addressing - Used in memory access instructions. Memory address calculated by adding the 16 bit Imm to the base address in rs.
Example: sw $s3, 4($s0)

→ 101011 | 00000 | 10011 | 0000 0000 0000 0100

sw $s0 $s3 4

This stores the contents of $s3 in the 4-bit word starting in memory location ($s0 + 4) + 4
(sw = Store word → address must be a multiple of 4)

PC-relative – used by branching instructions. If branching goes backwards, the imm is negative (two's complement)
(eg bne, beq - NOT j)

Pseudo-Direct – used by j-type → converting between 32 bit memory addresses and the 26 bit addr in the instruction.

AVR

32 general purpose 8-bit registers
- R26 - R31 are treated as three 16-bit registers.

Status Register – separate 8-bit register - each bit is a flag

Harvard architecture – Separate program and data memory

Three I/O ports – B, C and D.
- Direction of each pin of the ports determined by DDRB/C/D – 1 is output, 0 is input.
  Instructions SBI (set bit) and CBI (clear bit) used to update the values, e.g.:

```
SBI DDRB, 1
```

  sets the pin PB1 to output.
- Input values continuously recorded on I/O registers PINB, PINC, PIND, with each bit corresponding to a pin on the ports.
- Output values can be set using PORTB, PORTC, PORTD I/O registers, each bit corresponding to a pin, e.g.

```
SBI PORTB, 1
```

  sets the output of pin PB1 to 1.

Some instructions:

SBIC <PIN B/C/D, 0-7> → skip next instruction if specified input bit is 0.

LDI <R>, <num> → load a value into a register.

RJMP <Label> → jump

SUBI <R>, <num> → subtract a value from the value in a register. If result is 0, set Z flag in SR to 1, otherwise clear it.

BRNE <label> → branches iff Z bit is 0 → use in conjunction with SUBI.

RET → return

.equ <identifier>, <value> → initialise named constants.